



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

DIPARTIMENTO DI MATEMATICA
LAUREA TRIENNALE IN INFORMATICA

PROGETTO DI BASI DI DATI

Basi di Dati di un Sistema Informativo per la Gestione di un torneo del
GCC Pokémon

Alessandro Adami, Alberto José Toniolo

Indice

1	Abstract	2
2	Analisi dei Requisiti *DA FARE*	2
3	Progettazione Concettuale //RICORDARSI NUMERO PAGINA	3
4	Progettazione Logica	3
4.1	Analisi delle Ridondanze	5
4.2	Eliminazioni delle generalizzazioni	5
4.3	Schema Relazionale	7
5	Implementazione in PostgreSQL e Definizione delle Query	8
5.1	Definizione delle Query	8

1 Abstract

Questo lavoro sviluppa una base di dati progettata per gestire in modo strutturato e coerente le informazioni relative a un torneo, tenendo conto degli utenti, mazzi usati e eventuali carte che li compongono, restrizioni applicate nel torneo, la piattaforma su cui è realizzato e i risultati dei match. L'obiettivo principale è fornire un sistema che supporti le attività di monitoraggio delle statistiche (percentuale di utilizzo, percentuale di vittoria, ecc.) per ogni carta, fornendo informazioni sul bilanciamento del gioco nel suo complesso, oltre a tenere conto della performance di ogni partecipante (gruppo o individuale).

Nel contesto del torneo, la base di dati distingue l'utente tra squadre e i solitari, gestendo separatamente le informazioni relative a un'eventuale organizzazione eSports, oppure il biglietto per il singolo.

2 Analisi dei Requisiti *DA FARE*

Questa sezione riassume i requisiti a cui deve sottostare la base di dati.

Ospedali. Ogni ospedale è identificato da un codice univoco e contiene le seguenti informazioni:

- Codice univoco che distingue in modo univoco ogni struttura sanitaria.
- Numero di dipendenti (se noto).
- La regione geografica in cui si trova.

Gli ospedali si suddividono in: pubblici e privati.

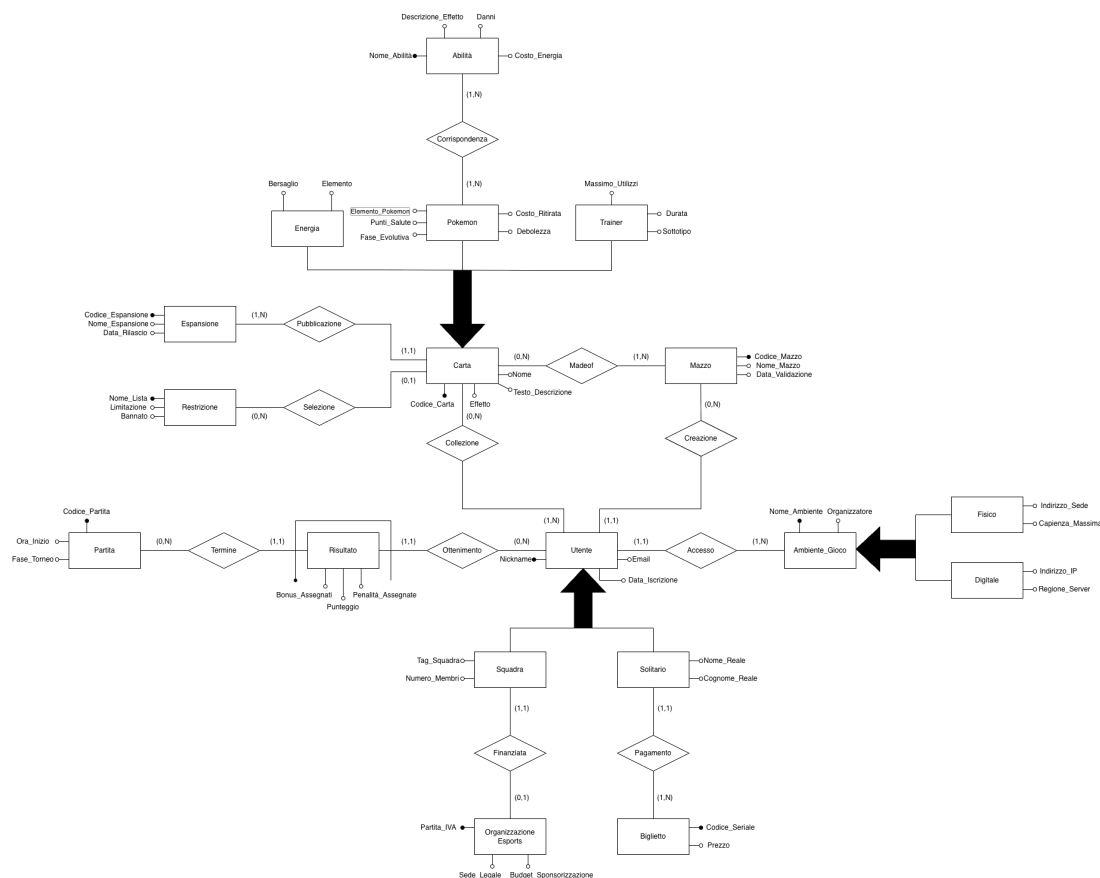


Figura 1: Diagramma E-R della base di dati relativa alla gestione di un torneo.

3 Progettazione Concettuale //RICORDARSI NUMERO PAGINA

Figura 1 mostra il diagramma Entità-Relazione (E-R) che riassume i requisiti descritti in sezione 2.

La tabella 1 a pagina 6 riassume tutte le entità e relazioni individuate dalla analisi dei requisiti nel diagramma E-R, con i relativi attributi rilevanti dall'analisi. Per le entità, viene anche fornito l'identificatore, che contiene anche relazioni nei tre casi di Reparto, Parto e Ricovero.

Il presente schema E-R non permette di rappresentare direttamente i seguenti vincoli:

- Una carta può essere bannata solo una volta
- L'attributo *Costo_Energia* in Abilità non può essere minore di 0
- L'attributo *Durata* in Trainer ha come valore minimo 1 (dura almeno un turno)
- Per ogni partita, il numero minimo di giocatori è 1 mentre il massimo è 2, ovvero:

$$\forall P \in \text{Partita}, \quad \exists U \in \text{Utente} : \quad 1 \leq |U| \leq 2$$

4 Progettazione Logica

In questa sezione viene illustrato il processo di "traduzione" dello schema concettuale in uno schema logico, con l'obiettivo di rappresentare i dati in modo preciso ed efficiente. Il primo passo consiste nell'analizzare le eventuali ridondanze nel modello, al fine di ottimizzare la struttura complessiva. Successivamente, si procede con l'eliminazione delle tre generalizzazioni. Infine, viene presentato il diagramma ristrutturato, con una descrizione delle modifiche apportate.

Entità	Descrizione	Attributi	Identificatore
Carta	Carta Generica	<i>Codice_Carta, Effetto, Testo_Descrizione, Nome</i>	<i>Codice_Carta</i>
Mazzo	Mazzo di carte	<i>Codice_Mazzo, Nome_Mazzo, Data_Validazione</i>	<i>Codice_Mazzo</i>
Espansione	Espansione di provenienza delle carte	<i>Codice_Espansione, Nome_Espansione, Data_Rilascio</i>	<i>Codice_Espansione, Nome_Espansione</i>
Restrizione	Restrizione dell'uso di una carta	<i>Nome_Lista, Limitazione, Bannato</i>	<i>Nome_Lista</i>
Pokemon	Carta Pokémon	<i>Elemento_Pokemon, Punti_Salute, Fase_Evolutiva, Costo_Ritirata, Debolezza</i>	
Abilità	Abilità di un Pokémon	<i>Nome_Abilità, Descrizione_Effetto, Danni, Costo_Energia</i>	<i>Nome_Abilità</i>
Energia	Carta Energia	<i>Bersaglio, Elemento</i>	
Trainer	Carta Trainer	<i>Massimo_Utilizzi, Durata, Sottotipo</i>	
Utente	Utente che gioca	<i>Nickname, Email, Data_Iscrizione</i>	<i>Nickname</i>
Squadra	Squadra di utenti	<i>Tag_Squadra, Numero_Membri</i>	
Organizzazione Esports	Organizzazione per squadre	<i>Partita_IVA, Sede_Legale, Budget_Sponsorizzazione</i>	<i>Partita_IVA</i>
Solitario	Utente che gioca da solo	<i>Nome_Reale, Cognome_Reale</i>	
Biglietto	Biglietto d'iscrizione al torneo	<i>Codice_Seriale, Prezzo</i>	<i>Codice_Seriale</i>
Risultato	Risultato alla fine di un match	<i>Codice_Partita, Punteggio, Bonus_Assignati, Penalità_Assegnate</i>	<i>Codice_Partita, Relazione "Termine"</i>
Partita	Partita singola di un match	<i>Codice_Partita, Ora_Inizio, Fase_Torneo</i>	<i>Codice_Partita</i>
Ambiente_Gioco	Ambiente in cui si svolge il torneo	<i>Nome_Ambiente, Organizzatore</i>	<i>Nome_Ambiente</i>
Fisico	Torneo in un posto fisico	<i>Indirizzo_Sede, Capacità_Massima</i>	
Digitale	Torneo su una piattaforma Online	<i>Indirizzo_IP, Regione</i>	

(a) Entità

Tabella 1: Dizionario dei dati.

Relazione	Descrizione	Componente	Attributi
Corrispondenza	Corrispondenza tra un Pokémon e la sua abilità posseduta	Pokemon, Abilità	
Part-Of	Appartenenza di una Carta ad un Mazzo	Carta, Espansione	
Selezione	Selezione di una carta dentro la lista delle restrizioni	Carta, Restrizione	
Collezione	Collezione di carte appartenenti ad un utente	Utente, Carta	
Creazione	Creazione di un mazzo da parte di un utente	Utente, Mazzo	
Finanziata	Organizzazione Esports finanzia la Squadra	Squadra, Organizzazione Esports	
Pagamento	Utente singolo paga il biglietto di iscrizione	Solitario, Biglietto	
Ottenimento	Utente ottiene un risultato a fine match	Utente, Risultato	
Termine	Al termine di una partita si ottiene un risultato	Risultato, Partita	
Accesso	Utente accede al torneo che si realizza in un ambiente di gioco	Utente, Ambiente_Gioco	

(b) Relazioni

Tabella 1: Dizionario dei dati (continua).

4.1 Analisi delle Ridondanze

L'attributo N_RICOVERI in REPARTO, che memorizza il numero di pazienti ricoverati nel reparto presenta una ridondanza.

Concetto	Costrutto	Volume
REPARTO	E	50
OSPEDALIZZAZIONE	R	20000
RICOVERO	E	20000

Tabella 2: Assunzione dei volumi nella base di dati

4.2 Eliminazioni delle generalizzazioni

Le generalizzazioni descritte in Sezione 3 vengono eliminate attraverso una ristrutturazione dello schema concettuale, con l'obiettivo di semplificare la successiva implementazione del modello relazionale e ridurre la presenza di valori nulli. Le modifiche vengono applicate come segue:

- **CARTA:** La generalizzazione di Carta viene rimossa includendo le relazioni Is-Poke, Is-Ene e Is-Train, come mostrato in figura 2. Questa scelta riduce la presenza di valori nulli:

- **AMBIENTE GIOCO:** In maniera analoga alla generalizzazione di carta, per Ambiente_Gioco la generalizzazione viene rimossa introducendo le relazioni Is-Fis e Is-Dig. Nello stesso modo mantenendo un'unica entità avremmo valori nulli come Indirizzo_Ip e Regione_Server quando il contesto del torneo è fisico (e viceversa). Sarebbe stato possibile rimuovere l'entità Ambiente_Gioco incorporando gli attributi nelle entità figlie, ma avrebbe anche causato la necessità di duplicare la relazione Part-Of in due relazioni.
- **UTENTE:** Anche per Utente la generalizzazione viene rimossa introducendo due relazioni, Is-Squad e Is-Solo. Unendo tutti gli attributi sotto una singola entità Utente porterebbe a campi nulli come Tag_Squadra e Numero_Membri quando ci si riferisce a un utente solitario (e viceversa). Analogamente agli altri due casi, sarebbe stato possibile incorporare gli attributi nelle unità figlie, ma anche qui avremmo duplicato la relazione Part-Of nelle due relazioni, verso Squadra e Solitario.



4.3 Schema Relazionale

Lo schema ristrutturato contiene solamente costrutti mappabili in corrispettivi dello schema relazionale detto anche schema logico. L'asterisco dopo il nome degli attributi indica quelli che ammettono valori nulli.

- **Ambiente_Gioco**(Nome_Ambiente, Organizzatore)
- **Fisico**(Ambiente, Indirizzo_Sede, Capienza_Massima)
 - Fisico.Ambiente → Ambiente_Gioco.Nome_Ambiente
- **Digitale**(Ambiente, Indirizzo_IP, Regione_Server)
 - Digitale.Ambiente → Ambiente_Gioco.Nome_Ambiente
- **Mazzo**(Codice_Mazzo, Nome_Mazzo, Data_Validazione*)
- **Espansione**(Codice_Espansione, Nome_Espansione, Data_Rilascio)
- **Restrizione**(Nome_Lista, Limitazione, Bannato)
- **Utente**(Nickname, Email, Data_Iscrizione, Ambiente*, Mazzo*)
 - Utente.Ambiente → Ambiente_Gioco.Nome_Ambiente
 - Utente.Mazzo → Mazzo.Codice_Mazzo
- **Biglietto**(Codice_Seriale, Prezzo*)
- **Solitario**(Utente, Nome_Reale, Cognome_Reale, Biglietto*)
 - Solitario.Utente → Utente.Nickname
 - Solitario.Biglietto → Biglietto.Codice_Seriale
- **Organizzazione_Esports**(Partita_IVA, Sede_Legale, Budget_Sponsorizzazione, Squadra*)
- **Squadra**(Utente, Tag_Squadra, Numero_Membri*, Esports*)
 - Squadra.Utente → Utente.Nickname
 - Squadra.Esports → Organizzazione_Esports.Partita_IVA
- **Carta**(Codice_Carta, Nome, Testo_Descrizione, Effetto*, Espansione*, Restrizione*)
 - Carta.Espansione → Espansione.Codice_Espansione
 - Carta.Restrizione → Restrizione.Nome_Lista
- **Pokemon**(Carta, Elemento_Pokemon, Punti_Salute, Fase_Evolutiva, Debolezza, Costo_Ritirata)
 - Pokemon.Carta → Carta.Codice_Carta
- **Trainer**(Carta, Massimo_Utilizzi*, Sottotipo, Durata)
 - Trainer.Carta → Carta.Codice_Carta
- **Energia**(Carta, Bersaglio, Elemento)
 - Energia.Carta → Carta.Codice_Carta

- **Abilità**(Nome__Abilità, Descrizione__Effetto, Danni*, Costo__Energia*)
- **Partita**(Codice__Partita, Ora__Inizio*, Fase__Torneo)
- **Risultato**(Utente, Partita, Punteggio, Bonus__Assegnati*, Penalità__Assegnate*)
 - Risultato.Utente → Utente.Nickname
 - Risultato.Partita → Partita.Codice__Partita

5 Implementazione in PostgreSQL e Definizione delle Query

Il file collezionismo.sql contiene il codice SQL necessario per la creazione e il popolamento delle tabelle del database. Questo file include inoltre una serie di query per l'estrazione dei dati e un indice creato specificamente per migliorare le prestazioni di una di queste interrogazioni.

5.1 Definizione delle Query

Query 1: Contare il numero di reparti per ciascun ospedale stampandoli in ordine decrescente.

```
1 SELECT O.Codice AS CodiceOspedale, COUNT(R.Codice) AS NumeroReparti
2 FROM Ospedale O
3 JOIN Reparto R ON O.Codice = R.Ospedale
4 GROUP BY O.Codice
5 ORDER BY NumeroReparti DESC;
```

Estratto dell'output:

codiceospedale (integer)	numeroreparti (bigint)
1	75
2	4
3	2
4	11